

Assignment – 5

Submitted by ~ Ishika Dutta (CT_CSI_DV_5572)

Task 5.1

Create a Kubernetes cluster using minikube

Install Minikube for Linux

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64  
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

Start Minikube

```
minikube start
```

Verify cluster

```
kubectl get nodes
```

Other useful commands

Check cluster status

```
minikube status
```

Open dashboard

```
minikube dashboard
```

Stop cluster

```
minikube stop
```

Delete cluster

```
minikube delete
```

Task 5.2

Create a Kubernetes cluster using kubeadm

NOTE:

Disable swap:

```
sudo swapoff -a  
sudo sed -i '/ swap / s/^#/' /etc/fstab
```

Set hostname:

```
sudo hostnamectl set-hostname master-node
```

Configure firewall / networking (open ports):

Component	Port	Protocol
kube-apiserver	6443	TCP
etcd	2379-2380	TCP
kubelet	10250	TCP

Enable bridge networking:

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
br_netfilter
EOF

sudo modprobe br_netfilter

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
EOF

sudo sysctl --system
```

Install container runtime

```
sudo apt-get update
sudo apt-get install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo systemctl restart containerd
sudo systemctl enable containerd
```

Install kubeadm, kubelet, kubectl

```
sudo apt-get update && sudo apt-get install -y apt-transport-https ca-certificates curl

curl -fsSL https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add -

cat <<EOF | sudo tee /etc/apt/sources.list.d/kubernetes.list
deb https://apt.kubernetes.io/ kubernetes-xenial main
EOF

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

Initialize master node

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16
```

Setup kubeconfig for your user:

```
mkdir -p $HOME/.kube  
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config  
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Install a CNI (network plugin)

```
kubect1 apply -f https://raw.githubusercontent.com/coreos/flannel/master/Documentation/kube-flannel.yml
```

Join worker nodes

```
kubeadm join <master-ip>:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

Verify the cluster

```
kubect1 get nodes
```

Task 5.3

Deploy an AKS cluster using the portal. Access the dashboard and create roles for multiple users

Create AKS Cluster via Azure Portal

- In Azure Portal Search for: Kubernetes services → + Create → Create Kubernetes cluster
- Basics tab
 - Subscription: Select your Azure subscription
 - Resource Group: Create new (e.g. aks-rg)
 - Cluster name: e.g. myAKSCluster
 - Region: e.g. Central India
 - Kubernetes version: Choose latest stable

- *Node pools*
 - *Configure size (e.g. Standard_DS2_v2)*
 - *Node count (e.g. 3)*
- *Authentication*
 - *Enable RBAC*
 - *Enable Azure AD integration (optional for advanced access control)*
- *Networking*
 - *Choose networking (Azure CNI / Kubenet)*
 - *Leave defaults unless you need advanced setup*
- *Monitoring*
 - *Enable Container Insights*
- *Review + Create*
 - *Click Review + Create*
 - *Click Create*

Access Kubernetes Dashboard

Install dashboard:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/v2.7.0/aio/deploy/recommended.yaml
```

Access dashboard (with proxy):

```
az aks get-credentials --resource-group aks-rg --name myAKSCluster
kubectl proxy
```

Open:

```
http://localhost:8001/api/v1/namespaces/kubernetes-dashboard/services/https:kubernetes-dashboard:/proxy/
```

Log in:

```
kubectl create serviceaccount dashboard-admin-sa
kubectl create clusterrolebinding dashboard-admin-sa --clusterrole=cluster-admin --serviceaccount=default:dashboard-admin-sa
```

Get token:

```
kubectl get secret $(kubectl get sa dashboard-admin-sa -o jsonpath="{.secrets[0].name}") -o go-template="{{.data.token | base64decode}}"
```

Create Roles for Multiple Users (RBAC)

Create namespace

```
kubectl create namespace team-a
```

Create Role

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: team-a
  name: team-a-reader
rules:
- apiGroups: [""]
  resources: ["pods", "services"]
  verbs: ["get", "list", "watch"]
```

Apply:

```
kubectl apply -f team-a-role.yaml
```

Create RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: team-a-reader-binding
  namespace: team-a
subjects:
- kind: User
  name: user1@yourdomain.com # Replace with Azure AD user
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: team-a-reader
  apiGroup: rbac.authorization.k8s.io
```

Apply:

```
kubectl apply -f team-a-binding.yaml
```

Task 5.4

Deploy a microservice application on AKS cluster and access it using public internet

Connect to Cluster:

```
az aks get-credentials --resource-group <your-rg> --name <your-aks-cluster>
```

Create Deployment:

Create a microservice deployment (e.g. k8s/echo-server):

```
kubectl create deployment hello-app --image=k8s.gcr.io/echoserver:1.4
```

Expose via LoadBalancer

```
kubectl expose deployment hello-app --port=80 --target-port=8080 --type=LoadBalancer
```

Get public IP

```
kubectl get service hello-app
```

Clean up

```
kubectl delete service hello-app  
kubectl delete deployment hello-app
```

Task 5.5

Expose services in the cluster with node port, cluster IP, load balancer

Example Deployment

Let's assume you have a deployment:

```
kubectl create deployment hello-app --image=k8s.gcr.io/echoserver:1.4
```

Expose using different service types

ClusterIP (default)

```
kubectl expose deployment hello-app --port=80 --target-port=8080 --type=ClusterIP
```

View:

```
kubectl get svc hello-app
```

NodePort

```
kubectl expose deployment hello-app --port=80 --target-port=8080 --type=NodePort
```

LoadBalancer

```
kubectl expose deployment hello-app --port=80 --target-port=8080 --type=LoadBalancer
```